KNODE-Cosserat: Hybrid Physics+NN Surrogate

for Snake Robot Cosserat Rod Dynamics

Snake-HRL Project — Research Report

March 2026

1 Introduction and Motivation

The pure MLP surrogate (system-formulation.tex, Section 6) replaces the full Verlet integration pipeline — all $\sim$377,000 scalar operations per RL step — with a single forward pass $f_\theta\colon \mathbb{R}^{131} \to \mathbb{R}^{124}$. While this achieves $\sim$17,000$\times$ throughput improvement, it discards all physics structure and must learn the entire transition operator from data.

The **KNODE-Cosserat** (Knowledge-based Neural ODE) approach takes a different strategy: embed the known physics structure into the model and train a neural network to correct only the expensive or inaccurate parts. The key insight is that many stages of the Verlet substep are *cheap and exact* — the neural network need not learn them at all.

1.1 Computational Cost Decomposition

From the equation count (system-formulation.tex, Table 1), the per-substep cost distributes as:

Stage	Ops/substep	Cheap?	NN replaces?
Stage 1: Half-step positions	62	Yes	No
Stage 2a: Geometry update	60	Yes	No
Stage 2b: Shear–stretch forces	∼120	**No**	**Yes**
Stage 2c: Bending–twist torques	∼200	**No**	**Yes**
Stage 2d: External forces	126	Yes	No
Stage 2e: Accelerations	62	Yes	No
Stage 2f: Velocity update	62	Yes	No
Stage 3: Full-step positions	62	Yes	No
Total	∼754		
NN-replaced	∼320		**42%**

Table 1: Per-substep cost decomposition. Stages 2b–2c (internal force assembly) account for 42% of the computation and involve the most complex nonlinear operations (constitutive laws, cross products, curvature assembly). The remaining 58% consists of cheap linear operations.

The hybrid approach computes Stages 1, 2a, 2d–2f, and 3 analytically (they are cheap matrix–vector operations) and replaces only Stages 2b–2c with a neural network.

2 The KNODE-Cosserat Framework

2.1 Core Equation

The KNODE framework [1] augments a known (possibly imperfect) physics model with a learned residual:

$$\dot{\mathbf{x}} = f_{\text{physics}}(\mathbf{x}, t) + g_\theta(\mathbf{x}, t) \tag{1}$$

where f_{physics} encodes the known Cosserat rod dynamics and g_θ is a neural network that learns the model error.

2.2 Adaptation to Our Problem

Rather than correcting spatial derivatives as in the original KNODE-Cosserat paper (which targets a tendon-driven continuum arm), we adapt the framework to our *temporally-discretized* Position Verlet integration of the planar (2D) Cosserat rod.

The key decomposition is:

$$\boxed{\mathbf{F}_i^{\text{total}} = \underbrace{h_\theta(\mathbf{q}^{n+1/2}, \dot{\mathbf{q}}^n, \mathbf{a}_t)}_{\text{learned internal forces (Stages 2b–2c)}} + \underbrace{\mathbf{f}_i^{\text{grav}} + \mathbf{f}_i^{\text{rft}} + \mathbf{f}_i^{\text{damp}}}_{\text{analytical external forces (Stage 2d)}}} \tag{2}$$

where $h_\theta \colon \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{62}$ predicts the combined internal forces ($\mathbb{R}^{42}$) and internal torques ($\mathbb{R}^{20}$) that would be assembled by the full constitutive-law pipeline.

2.3 What the NN Learns vs. What Physics Computes

Operation	Computed by	Eq. ref
Half-step positions	Physics	SF§4.1
Tangent vectors, lengths, dilatation	Physics	SF§4.2a
Strain computation ($\boldsymbol{\sigma}$)	—	—
Constitutive law (shear–stretch)	NN	SF§4.2b
Curvature assembly ($\boldsymbol{\kappa}$)	—	—
Constitutive law (bend–twist)	NN	SF§4.2c
Internal torque (5-term assembly)	NN	SF§4.2c
Gravity, RFT friction, damping	Physics	SF§4.2d
Mass-normalized acceleration	Physics	SF§4.2e
Velocity update	Physics	SF§4.2f
Full-step positions	Physics	SF§4.3

Table 2: Responsibility split. The NN replaces the internal force/torque assembly (Steps 2b–2c) while all kinematic updates, external forces, and time integration remain analytical.

3 Architecture

3.1 Internal Force Surrogate Network

The NN h_θ replaces the constitutive law and internal force assembly:

$$h_\theta \colon \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{62}, \qquad d_{\text{in}} = 62 + 62 + 5 + 2 = 131 \tag{3}$$

Input features ($d_{\text{in}} = 131$):

- Half-step positions $\mathbf{q}^{n+1/2} \in \mathbb{R}^{62}$ (21 nodes $\times$ 2 components + 20 yaw angles)
- Current velocities $\dot{\mathbf{q}}^n \in \mathbb{R}^{62}$ (21 nodes $\times$ 2 components + 20 angular velocities)

- Action $\mathbf{a}_t \in \mathbb{R}^5$ (CPG parameters)
- Phase encoding $[\sin(\omega t), \cos(\omega t)] \in \mathbb{R}^2$

Output ($d_{\text{out}} = 62$):

- Internal forces $\mathbf{F}^{\text{int}} \in \mathbb{R}^{42}$ (21 nodes $\times$ 2 components)
- Internal torques $\mathbf{m}^{\text{int}} \in \mathbb{R}^{20}$ (20 elements)

Architecture:

$$h_\theta = \text{Linear}(131, 256) \to [\text{Linear}(256, 256) + \text{LayerNorm} + \text{SiLU}]^{\times 2} \to \text{Linear}(256, 62) \quad (4)$$

Output layer is zero-initialized so the model initially predicts zero internal forces (rest configuration). Total parameters: $\sim$135K (vs. $\sim$654K for the full-state MLP surrogate).

3.2 Alternative: Residual Correction on Simplified Physics

Instead of replacing Stages 2b–2c entirely, a residual variant uses a *simplified* physics backbone and corrects its errors:

$$\mathbf{F}^{\text{int}} = \mathbf{F}^{\text{int}}_{\text{simplified}} + \delta_\theta(\mathbf{q}^{n+1/2}, \dot{\mathbf{q}}^n) \quad (5)$$

where $\mathbf{F}^{\text{int}}_{\text{simplified}}$ uses linearized bending (small-angle approximation), drops the unsteady dilatation and Lagrangian transport terms from (SF Eq. 16), and uses a coarser spatial stencil. The correction network δ_θ then only needs to learn:

- Nonlinear bending effects (large-curvature corrections)
- Missing torque terms (transport, unsteady dilatation)
- Discretization errors

This further reduces what the NN must learn, at the cost of implementing and maintaining a simplified physics model.

4 Algorithms

4.1 Hybrid Verlet Substep

Algorithm 1 KNODE-Cosserat Hybrid Verlet Substep

Input: State $(\mathbf{q}^n, \dot{\mathbf{q}}^n) \in \mathbb{R}^{124}$, action $\mathbf{a}_t \in \mathbb{R}^5$, time t, trained network h_θ

Output: Next state $(\mathbf{q}^{n+1}, \dot{\mathbf{q}}^{n+1}) \in \mathbb{R}^{124}$

— Stage 1: Kinematic half-step (analytical) —

$\mathbf{x}^{n+1/2} \leftarrow \mathbf{x}^n + \frac{1}{2}\Delta t\, \dot{\mathbf{x}}^n$ ▷ Node positions

$\boldsymbol{\psi}^{n+1/2} \leftarrow \boldsymbol{\psi}^n + \frac{1}{2}\Delta t\, \boldsymbol{\omega}^n$ ▷ Element yaw angles

— Stage 2a: Geometry update (analytical) —

$\Delta\mathbf{x} \leftarrow \mathbf{D}\,\mathbf{x}^{n+1/2}$ ▷ Edge vectors via difference operator

$\boldsymbol{\ell} \leftarrow \|\Delta\mathbf{x}\|_{\text{row}}$ ▷ Element lengths

$\boldsymbol{\varepsilon} \leftarrow \boldsymbol{\ell} \oslash \boldsymbol{\ell}^{\text{rest}}$ ▷ Dilatation

$\mathbf{t} \leftarrow \Delta\mathbf{x} \oslash \boldsymbol{\ell}$ ▷ Unit tangent vectors

— Stages 2b–2c: Internal forces via neural network —

$\mathbf{z}_{\text{phase}} \leftarrow [\sin(\omega t),\ \cos(\omega t)]$ ▷ Phase encoding

$\mathbf{z}_{\text{in}} \leftarrow [\bar{\mathbf{q}}^{n+1/2} \parallel \bar{\dot{\mathbf{q}}}^n \parallel \mathbf{a}_t \parallel \mathbf{z}_{\text{phase}}]$ ▷ Concatenate, z-normalize

$[\mathbf{F}^{\text{int}},\ \mathbf{m}^{\text{int}}] \leftarrow h_\theta(\mathbf{z}_{\text{in}})$ ▷ $\mathbb{R}^{131} \to \mathbb{R}^{62}$

— Stage 2d: External forces (analytical) —

$\mathbf{F}^{\text{grav}} \leftarrow \mathbf{M}\,\mathbf{g}$ ▷ Gravity

$\mathbf{v}_t \leftarrow (\mathbf{t} \cdot \dot{\mathbf{x}})\,\mathbf{t}, \quad \mathbf{v}_n \leftarrow \dot{\mathbf{x}} - \mathbf{v}_t$ ▷ Velocity decomposition

$\mathbf{F}^{\text{rft}} \leftarrow -c_t\,\mathbf{v}_t - c_n\,\mathbf{v}_n$ ▷ Anisotropic RFT friction

$\mathbf{F}^{\text{damp}} \leftarrow -\gamma\,\dot{\mathbf{x}}$ ▷ Numerical damping

— Stage 2e–f: Acceleration and velocity update (analytical) —

$\ddot{\mathbf{x}} \leftarrow \mathbf{M}^{-1}(\mathbf{F}^{\text{int}} + \mathbf{F}^{\text{grav}} + \mathbf{F}^{\text{rft}} + \mathbf{F}^{\text{damp}})$

$\dot{\boldsymbol{\omega}} \leftarrow \mathbf{J}_{\text{eff}}^{-1}\,\mathbf{m}^{\text{int}}$ ▷ Angular acceleration from NN torques

$\dot{\mathbf{x}}^{n+1} \leftarrow \dot{\mathbf{x}}^n + \Delta t\,\ddot{\mathbf{x}}$

$\boldsymbol{\omega}^{n+1} \leftarrow \boldsymbol{\omega}^n + \Delta t\,\dot{\boldsymbol{\omega}}$

— Stage 3: Full-step positions (analytical) —

$\mathbf{x}^{n+1} \leftarrow \mathbf{x}^{n+1/2} + \frac{1}{2}\Delta t\,\dot{\mathbf{x}}^{n+1}$

$\boldsymbol{\psi}^{n+1} \leftarrow \boldsymbol{\psi}^{n+1/2} + \frac{1}{2}\Delta t\,\boldsymbol{\omega}^{n+1}$

return $(\mathbf{q}^{n+1}, \dot{\mathbf{q}}^{n+1})$

4.2 Multi-Substep RL Transition

Algorithm 2 KNODE-Cosserat RL Environment Step

Input: State $\mathbf{s}_t \in \mathbb{R}^{124}$, action $\mathbf{a}_t \in \mathbb{R}^5$, number of substeps n_{sub}, trained network h_θ
Output: Next state $\mathbf{s}_{t+1} \in \mathbb{R}^{124}$, reward r_t, observation $\mathbf{o}_{t+1}$

```
(q^0, q̇^0) ← s_t                                          ▷ Unpack positions and velocities
a_phys ← MapAction(a_t)                                   ▷ [−1, 1]^5 → (A, f, k, φ, b)
for k = 0, 1, ..., n_sub − 1 do
    t_k ← t + k · Δt
    (q^{k+1}, q̇^{k+1}) ← HybridVerletSubstep(q^k, q̇^k, a_t, t_k, h_θ)   ▷ Alg. 1
end for
s_{t+1} ← (q^{n_sub}, q̇^{n_sub})
o_{t+1} ← ExtractObservation(s_{t+1})                     ▷ FFT features, CoM, heading, etc.
r_t ← ComputeReward(s_t, s_{t+1}, a_t)
return (s_{t+1}, r_t, o_{t+1})
```

Key difference from the pure MLP surrogate: the MLP surrogate replaces the entire n_{sub}-substep composition with a single forward pass $f_\theta(\mathbf{z}_t) \approx T(\mathbf{s}_t, \mathbf{a}_t) - \mathbf{s}_t$. The KNODE approach retains the Verlet time-stepping structure and calls the NN *once per substep*, yielding n_{sub} NN evaluations per RL step. This preserves the symplectic integration structure at the cost of $n_{\text{sub}}\times$ more NN forward passes.

4.3 Training the Internal Force Network

Algorithm 3 Training the Internal Force Network h_θ

Input: Dataset $\mathcal{D} = \{(\mathbf{q}_i^{n+1/2}, \dot{\mathbf{q}}_i^n, \mathbf{a}_i, t_i, \mathbf{F}_i^{\text{int}}, \mathbf{m}_i^{\text{int}})\}_{i=1}^N$
Output: Trained network parameters θ^*

— Phase 0: Data collection from PyElastica —

for each transition in PyElastica rollouts **do**
  Run full Verlet substep; record intermediate quantities:
    $\mathbf{q}^{n+1/2}$ (half-step positions after Stage 1)
    $\mathbf{F}^{\text{int}}$ (internal forces from Stage 2b)
    $\mathbf{m}^{\text{int}}$ (internal torques from Stage 2c)
  Store $(\mathbf{q}^{n+1/2}, \dot{\mathbf{q}}^n, \mathbf{a}, t, \mathbf{F}^{\text{int}}, \mathbf{m}^{\text{int}})$ in $\mathcal{D}$
end for

— Phase 1: Single-step supervised training —

for epoch $= 1, \ldots, E_1$ **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $\hat{\mathbf{y}}_i \leftarrow h_\theta(\bar{\mathbf{q}}_i^{n+1/2}, \bar{\dot{\mathbf{q}}}_i^n, \mathbf{a}_i, \mathbf{z}_{\text{phase},i})$ for $i \in \mathcal{B}$
    $\mathcal{L}_{\text{single}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left\| \hat{\mathbf{y}}_i - [\mathbf{F}_i^{\text{int}} \,\|\, \mathbf{m}_i^{\text{int}}] \right\|^2$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{single}}$ ▷ Adam optimizer
  end for
end for

— Phase 2: Multi-step rollout fine-tuning —

for epoch $= E_1 + 1, \ldots, E_1 + E_2$ **do**
  for trajectory window $(\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_L)$ of length L **do**
    $\hat{\mathbf{s}}_0 \leftarrow \mathbf{s}_0$
    for $k = 0, \ldots, L-1$ **do**
      $\hat{\mathbf{s}}_{k+1} \leftarrow \textsc{HybridVerletSubstep}(\hat{\mathbf{s}}_k, \mathbf{a}_k, t_k, h_\theta)$ ▷ Autoregressive
    end for
    $\mathcal{L}_{\text{rollout}} \leftarrow \frac{1}{L} \sum_{k=0}^{L-1} \gamma^k \|\hat{\mathbf{s}}_{k+1} - \mathbf{s}_{k+1}^{\text{true}}\|^2$
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{single}} + \lambda_r \mathcal{L}_{\text{rollout}}$ ▷ $\lambda_r = 0.1$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ ▷ BPTT through L substeps
  end for
end for
return $\theta^* \leftarrow \theta$

Data collection note: Unlike the full-state MLP surrogate which collects $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})$ pairs, the KNODE approach requires *intermediate* quantities from within the Verlet substep — specifically the internal forces and torques computed at Stages 2b–2c. This requires instrumenting the PyElastica integration loop to expose these intermediate values.

4.4 Residual Correction Variant

Algorithm 4 Simplified-Physics + Residual Correction

Input: Half-step state $\mathbf{q}^{n+1/2}$, velocities $\dot{\mathbf{q}}^n$, geometry $(\boldsymbol{\ell}, \boldsymbol{\varepsilon}, \mathbf{t})$

Output: Internal forces $\mathbf{F}^{\text{int}}$ and torques $\mathbf{m}^{\text{int}}$

— Simplified physics backbone —

$\boldsymbol{\sigma} \leftarrow \mathbf{Q}_{\text{blk}}^T (\Delta\mathbf{x} \oslash \boldsymbol{\ell}^{\text{rest}})$ ▷ Strain (same as full model)

$\mathbf{n}^L \leftarrow \mathbf{S}_{\text{blk}} (\boldsymbol{\sigma} - \boldsymbol{\sigma}^{\text{rest}})$ ▷ Shear–stretch stress

$\mathbf{F}_{\text{simple}}^{\text{int}} \leftarrow \mathbf{D}^T [\text{diag}(\boldsymbol{\varepsilon}^{-1})\, \mathbf{Q}_{\text{blk}}^T\, \mathbf{n}^L]$ ▷ Full shear–stretch forces (exact)

$\boldsymbol{\kappa}_v \leftarrow \text{Curvature}(\mathbf{Q}_{\text{blk}})$ ▷ Curvature at Voronoi nodes

$\boldsymbol{\tau}_v^L \leftarrow \mathbf{B}_{\text{blk}} (\boldsymbol{\kappa}_v - \boldsymbol{\kappa}_v^{\text{rest}})$ ▷ Bending–twist stress

$\mathbf{m}_{\text{simple}}^{\text{int}} \leftarrow \mathbf{D}_v [\text{diag}(\hat{\boldsymbol{\varepsilon}}_v^{-3})\, \boldsymbol{\tau}_v^L]$ ▷ Bend–twist gradient only (term i)

— Neural residual correction —

$\mathbf{z}_{\text{in}} \leftarrow [\bar{\mathbf{q}}^{n+1/2} \parallel \bar{\dot{\mathbf{q}}}^n \parallel \mathbf{a}_t \parallel \mathbf{z}_{\text{phase}}]$

$[\delta\mathbf{F},\ \delta\mathbf{m}] \leftarrow \delta_\theta(\mathbf{z}_{\text{in}})$ ▷ $\mathbb{R}^{131} \to \mathbb{R}^{62}$

— Combined output —

$\mathbf{F}^{\text{int}} \leftarrow \mathbf{F}_{\text{simple}}^{\text{int}} + \delta\mathbf{F}$

$\mathbf{m}^{\text{int}} \leftarrow \mathbf{m}_{\text{simple}}^{\text{int}} + \delta\mathbf{m}$

return $(\mathbf{F}^{\text{int}}, \mathbf{m}^{\text{int}})$

The simplified backbone computes the shear–stretch forces exactly (they are relatively cheap) and approximates the bending–twist torques using only the first of five terms in (SF Eq. 16). The residual network δ_θ corrects for:

(i) The four dropped torque terms: curvature×couple, shear–stretch couple, Lagrangian transport, unsteady dilatation

(ii) Nonlinear effects at large curvature

(iii) Discretization artifacts from the staggered grid

5 Comparison of Approaches

	Full MLP Surrogate	**KNODE (Replace)**	**KNODE (Residual)**	**DD-PINN**	**PyElastica (Ground Truth)**
NN calls / RL step	1	n_{sub}	n_{sub}	1	0
NN input dim	131	131	131	varies	—
NN output dim	124	62	62	varies	—
NN parameters	∼654K	∼135K	∼135K	∼100K	—
Physics preserved	None	Kinematics, external forces	All of left + simplified 2b–c	PDE as loss	Exact
Training data (transitions)	∼1M transitions	∼100K substeps	∼10K substeps	∼1K collocation	—
Rollout stability	Low	Medium	High	High	Perfect
Throughput gain	∼17,000×	∼30×	∼20×	∼1,000×	1×
Implementation	Simple	Medium	Hard	Hard	Existing

Table 3: Comparison of surrogate approaches. The KNODE variants trade throughput for physical consistency and data efficiency. The throughput estimate for KNODE assumes $n_{\text{sub}} = 10$ (reduced from 500 by using a larger Δt enabled by the physics backbone).

6 Why Explicit Verlet Cannot Take Larger Timesteps

Critical observation: Replacing the force computation with a neural network does *not* change the stability limit of the explicit Verlet integrator.

The CFL condition for explicit symplectic integration depends on the eigenvalues of the force Jacobian $\partial \mathbf{F}/\partial \mathbf{q}$, which are governed by the rod's elastic stiffness:

$$\Delta t_{\max} \leq \frac{2}{\sqrt{\lambda_{\max}(\mathbf{M}^{-1}\mathbf{K})}} \tag{6}$$

where $\mathbf{K}$ is the effective stiffness matrix and $\lambda_{\max}$ is its largest eigenvalue. If the NN predicts forces with similar magnitudes to the true physics (as it must, to be accurate), the same stability limit applies. The 500 substeps at $\Delta t = 0.001\,\text{s}$ are a *stability requirement of explicit integration*, not a cost issue with force computation.

Consequence: The hybrid Verlet approach (Algorithms 1–2) requires **500 NN evaluations per RL step**. For a 135K-parameter MLP on GPU, this costs $\sim 500 \times 0.005\,\text{ms} \approx 2.5\,\text{ms}$ per environment — only a $\sim 20\times$ speedup over PyElastica (46 ms), and *not* the $17{,}000\times$ of the pure MLP surrogate.

This makes the explicit Verlet hybrid **impractical for RL training**. The solution is to switch to an *implicit* integrator, which is inherently stable at large timesteps.

7 DisMech: Implicit Euler Integration

DisMech (Discrete Elastic Rods) uses *implicit Euler* with Newton's method, taking $\Delta t = 0.05\,\mathrm{s}$ — $50\times$ larger than PyElastica. Each RL control step ($\Delta t_{\text{ctrl}} = 0.5\,\mathrm{s}$) requires only $n_{\text{sub}} = 10$ implicit steps, each solved by Newton iteration.

7.1 Implicit Euler Formulation

At each timestep, DisMech solves the nonlinear system:

$$\boxed{\mathbf{r}(\mathbf{q}^{n+1}) = \mathbf{M}\,\frac{\mathbf{q}^{n+1} - 2\mathbf{q}^{n} + \mathbf{q}^{n-1}}{\Delta t^2} + \left.\frac{\partial E}{\partial \mathbf{q}}\right|_{\mathbf{q}^{n+1}} - \mathbf{f}_{\text{ext}}(\mathbf{q}^{n+1}) = \mathbf{0}} \tag{7}$$

where $\mathbf{q} \in \mathbb{R}^{126}$ collects the 3D nodal positions and edge twist angles (21 nodes $\times$ 3 + 20 twists = 83, but in 2D this reduces to $\sim$62), and $E(\mathbf{q})$ is the total elastic energy (stretching + bending + twisting).

7.2 Newton Solver Loop

Each implicit step is solved iteratively:

$$\mathbf{J}(\mathbf{q}^{(k)})\,\Delta\mathbf{q} = -\mathbf{r}(\mathbf{q}^{(k)}), \qquad \mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} + \Delta\mathbf{q} \tag{8}$$

where $\mathbf{J} = \partial\mathbf{r}/\partial\mathbf{q}$ is the Jacobian (tangent stiffness matrix), assembled from the Hessian of the elastic energy plus mass and damping contributions. DisMech uses max_iter $= 25$, tolerance 10^{-4}, and typically converges in 3–8 iterations.

7.3 Where Neural Networks Can Be Injected

The Newton loop offers **four distinct injection points** for a neural network:

#	Injection Point	What the NN Does	Section
A	Warm-start initial guess	Predict $\mathbf{q}^{(0)} \approx \mathbf{q}^{n+1}$ to reduce Newton iterations	§8
B	Residual correction	Add $g_\theta(\mathbf{q}^{(k)})$ to the residual $\mathbf{r}$ at each Newton iteration	§9
C	Jacobian approximation	Replace or precondition $\mathbf{J}$ with a learned approximation	§10
D	Replace Newton entirely	Fixed-point implicit layer (DEQ-style)	§11

Table 4: Four hybrid architectures for DisMech's implicit Euler solver, ordered from least to most invasive modification of the solver internals.

8 Hybrid Architecture A: Amortized Warm-Start

The simplest hybrid: a neural network predicts a good initial guess for Newton's method, reducing the iteration count without modifying the solver.

8.1 Motivation

DisMech's Newton solver uses the previous timestep's solution $\mathbf{q}^n$ as the initial guess $\mathbf{q}^{(0)}$, requiring 3–8 iterations to converge. A learned warm-start can predict $\mathbf{q}^{(0)}$ closer to $\mathbf{q}^{n+1}$, potentially reducing iterations to 1–2.

8.2 Architecture

$$\mathbf{q}^{(0)} = \mathbf{q}^n + w_\theta(\mathbf{q}^n, \mathbf{q}^{n-1}, \mathbf{a}_t) \tag{9}$$

where $w_\theta \colon \mathbb{R}^d \to \mathbb{R}^{62}$ is a small MLP predicting the state increment.

Algorithm 5 Warm-Started Implicit Euler Step

Input: State $(\mathbf{q}^n, \mathbf{q}^{n-1})$, action $\mathbf{a}_t$, network w_θ

Output: Next state $\mathbf{q}^{n+1}$

— Neural warm-start —

$\mathbf{q}^{(0)} \leftarrow \mathbf{q}^n + w_\theta(\mathbf{q}^n, \mathbf{q}^{n-1}, \mathbf{a}_t)$ ▷ Predicted initial guess

— Standard Newton solve (unmodified DisMech) —

for $k = 0, 1, \ldots, K_{\max}$ **do**
  $\mathbf{r}^{(k)} \leftarrow \text{AssembleResidual}(\mathbf{q}^{(k)}, \mathbf{q}^n, \mathbf{q}^{n-1}, \mathbf{a}_t)$
  if $\|\mathbf{r}^{(k)}\| < \epsilon$ **then**
    break ▷ Converged
  end if
  $\mathbf{J}^{(k)} \leftarrow \text{AssembleJacobian}(\mathbf{q}^{(k)})$
  $\Delta\mathbf{q} \leftarrow -(\mathbf{J}^{(k)})^{-1}\,\mathbf{r}^{(k)}$ ▷ Linear solve
  $\mathbf{q}^{(k+1)} \leftarrow \mathbf{q}^{(k)} + \Delta\mathbf{q}$
end for
return $\mathbf{q}^{n+1} \leftarrow \mathbf{q}^{(K)}$

Advantages: No modification to DisMech internals. The Newton solver guarantees convergence to the correct solution regardless of NN quality. Training requires only $(\mathbf{q}^n, \mathbf{q}^{n-1}, \mathbf{a}_t, \mathbf{q}^{n+1})$ pairs from DisMech rollouts.

Expected speedup: If Newton iterations drop from 5 to 1–2, this is a 2.5–$5\times$ speedup per implicit step. At 10 steps per RL action, overall: 2.5–$5\times$ faster than DisMech. Literature reports 40–60% iteration reduction (Briden et al. 2024 [12]; Banerjee et al. 2025 [13]).

9 Hybrid Architecture B: Gray-Box Residual Correction

The NN adds a correction to the Newton residual at each iteration, injecting learned physics (e.g., unmodeled friction, muscle dynamics) into the solver.

9.1 Formulation

Modify the residual to include a learned term:

$$\mathbf{r}_{\text{hybrid}}(\mathbf{q}^{(k)}) = \mathbf{r}_{\text{physics}}(\mathbf{q}^{(k)}) + g_\theta(\mathbf{q}^{(k)}, \dot{\mathbf{q}}, \mathbf{a}_t) \tag{10}$$

The NN g_θ contributes to both the residual and (via autodiff) the Jacobian:

$$\mathbf{J}_{\text{hybrid}} = \mathbf{J}_{\text{physics}} + \frac{\partial g_\theta}{\partial \mathbf{q}} \tag{11}$$

The NN Jacobian $\partial g_\theta / \partial \mathbf{q}$ is computed by PyTorch autograd at each Newton iteration.

Algorithm 6 Gray-Box Newton Step with Residual Correction

Input: State $\mathbf{q}^{(k)}$ at Newton iteration k, action $\mathbf{a}_t$, network g_θ
Output: Updated state $\mathbf{q}^{(k+1)}$

— Physics residual and Jacobian —

$\mathbf{r}_{\text{phys}} \leftarrow \text{DisMech.AssembleResidual}(\mathbf{q}^{(k)})$
$\mathbf{J}_{\text{phys}} \leftarrow \text{DisMech.AssembleJacobian}(\mathbf{q}^{(k)})$

— Neural correction (with autodiff Jacobian) —

$\mathbf{q}_{\text{torch}} \leftarrow \text{torch.tensor}(\mathbf{q}^{(k)}, \text{requires_grad=True})$
$\mathbf{g} \leftarrow g_\theta(\mathbf{q}_{\text{torch}}, \dot{\mathbf{q}}, \mathbf{a}_t)$ ▷ NN forward pass
$\mathbf{J}_{\text{nn}} \leftarrow \text{torch.autograd.functional.jacobian}(g_\theta, \mathbf{q}_{\text{torch}})$

— Combined Newton step —

$\mathbf{r} \leftarrow \mathbf{r}_{\text{phys}} + \mathbf{g}$
$\mathbf{J} \leftarrow \mathbf{J}_{\text{phys}} + \mathbf{J}_{\text{nn}}$
$\Delta\mathbf{q} \leftarrow -\mathbf{J}^{-1}\,\mathbf{r}$ ▷ Linear solve
$\mathbf{q}^{(k+1)} \leftarrow \mathbf{q}^{(k)} + \Delta\mathbf{q}$

What the NN learns: Unmodeled forces — anisotropic RFT friction refinement, muscle actuation dynamics, contact forces during coiling, material nonlinearities.

Jacobian cost: For a 62-dimensional output, the full Jacobian $\partial g_\theta/\partial \mathbf{q} \in \mathbb{R}^{62\times 62}$ requires 62 backward passes (one per output dimension). At $\sim$135K parameters, each backward pass is $\sim$0.01 ms on GPU, so the full Jacobian costs $\sim$0.6 ms per Newton iteration. Over 3–5 iterations $\times$ 10 substeps: $\sim$18–30 ms per RL step.

This is directly inspired by Agarwal et al. (2024) [14], who demonstrate gray-box DNN components in implicit simulation with Newton-compatible Jacobians via autograd.

10 Hybrid Architecture C: Learned Preconditioner

Rather than modifying the residual, a learned preconditioner accelerates the linear solve $\mathbf{J}\,\Delta\mathbf{q} = -\mathbf{r}$ within each Newton iteration.

10.1 Formulation

Replace the direct solve with a preconditioned iterative solver:

$$\mathbf{P}_\theta^{-1}\,\mathbf{J}\,\Delta\mathbf{q} = -\mathbf{P}_\theta^{-1}\,\mathbf{r} \tag{12}$$

where $\mathbf{P}_\theta$ is a learned preconditioner that approximates $\mathbf{J}$. Chen (2024) [15] demonstrates GNN preconditioners for sparse linear systems that outperform ILU, achieving 40–60% iteration reduction for CG/GMRES.

For the 62-DOF rod Jacobian (banded/tridiagonal structure), the GNN takes the sparsity graph as input and outputs a sparse approximate inverse.

Practical consideration: At 62 DOFs, the linear solve via direct factorization (LU/Cholesky) costs $\sim$0.01 ms — already very cheap. A learned preconditioner is more impactful at larger DOF counts (>1000). For our problem, Architectures A and B are more practical.

11 Hybrid Architecture D: Deep Equilibrium (DEQ) Implicit Layer

The most radical approach: replace the entire Newton loop with a learned fixed-point iteration.

11.1 Formulation

The implicit Euler equation $\mathbf{r}(\mathbf{q}^{n+1}) = \mathbf{0}$ defines $\mathbf{q}^{n+1}$ as a fixed point. A Deep Equilibrium Model (DEQ) learns a contraction map F_θ whose fixed point approximates $\mathbf{q}^{n+1}$:

$$\mathbf{q}^* = F_\theta(\mathbf{q}^*, \mathbf{q}^n, \mathbf{a}_t), \qquad \mathbf{q}^{n+1} \approx \mathbf{q}^* \tag{13}$$

Forward pass: iterate $\mathbf{q}^{(k+1)} = F_\theta(\mathbf{q}^{(k)}, \mathbf{q}^n, \mathbf{a}_t)$ until convergence. Backward pass: use implicit differentiation (no need to store intermediate iterates), giving $O(1)$ memory.

Algorithm 7 DEQ Implicit Layer for Rod Dynamics

Input: State $\mathbf{q}^n$, action $\mathbf{a}_t$, DEQ network F_θ
Output: Next state $\mathbf{q}^{n+1}$

— Forward: fixed-point iteration —

$\mathbf{q}^{(0)} \leftarrow \mathbf{q}^n$ ▷ Initialize from previous state
for $k = 0, 1, \ldots, K_{\max}$ **do**
  $\mathbf{q}^{(k+1)} \leftarrow F_\theta(\mathbf{q}^{(k)}, \mathbf{q}^n, \mathbf{a}_t)$
  if $\|\mathbf{q}^{(k+1)} - \mathbf{q}^{(k)}\| < \epsilon$ **then**
    break
  end if
end for
$\mathbf{q}^* \leftarrow \mathbf{q}^{(K)}$

— Backward: implicit differentiation —

▷ Gradient via $\frac{d\mathbf{q}^*}{d\theta} = (I - \partial_{\mathbf{q}} F_\theta|_{\mathbf{q}^*})^{-1} \, \partial_\theta F_\theta|_{\mathbf{q}^*}$
▷ Computed by `torch.autograd` with custom backward hook
return $\mathbf{q}^{n+1} \leftarrow \mathbf{q}^*$

Key advantage: Automatic warm-starting. Consecutive rod configurations during smooth locomotion are very similar, so $\mathbf{q}^n$ is already close to $\mathbf{q}^{n+1}$ and the DEQ converges in 1–3 iterations. This property was demonstrated by Burger et al. (2025) [16] for molecular

dynamics force fields (10–20% speed improvement from temporal warm-starting).

Training: Requires $(\mathbf{q}^n, \mathbf{a}_t, \mathbf{q}^{n+1})$ pairs from DisMech. The DEQ learns to satisfy $\mathbf{q}^{n+1} = F_\theta(\mathbf{q}^{n+1}, \mathbf{q}^n, \mathbf{a}_t)$, enforced by implicit-layer backpropagation through the fixed point (ImPiNDiff, Akhare et al. 2025 [17]).

Risk: If F_θ is not a contraction, the fixed-point iteration diverges. Mitigation: enforce Lipschitz constraint $\|F_\theta\|_{\text{Lip}} < 1$ via spectral normalization.

11.2 Physics-Conditioned DEQ

A safer variant: embed known physics into the DEQ iteration:

$$\mathbf{q}^{(k+1)} = \mathbf{q}^n + \Delta t^2\, \mathbf{M}^{-1}\left[-\nabla E_{\text{simplified}}(\mathbf{q}^{(k)}) + g_\theta(\mathbf{q}^{(k)}, \mathbf{q}^n, \mathbf{a}_t)\right] \tag{14}$$

This combines a simplified elastic energy gradient (providing physically grounded iteration structure) with a learned correction term, similar to the KNODE concept but inside a fixed-point iteration rather than an ODE solve.

12 Additional Relevant Architectures

12.1 MENO: Matrix Exponential Neural Operator

Zanardi et al. (2025) [18] decompose stiff ODE systems into a linear time-varying part (handled by *matrix exponential*, which is A-stable) and a nonlinear residual (handled by a neural operator). For the Cosserat rod:

$$\mathbf{q}^{n+1} = \exp(\Delta t\, \mathbf{A}(\mathbf{q}^n))\, \mathbf{q}^n + \text{NeuralOp}_\theta(\mathbf{q}^n, \mathbf{a}_t) \tag{15}$$

where $\mathbf{A}$ is the linearized elastic stiffness matrix (banded, 62×62). At this size, the matrix exponential via Padé approximation costs $\sim$0.1 ms — much cheaper than Newton iteration — while providing unconditional A-stability.

12.2 Differentiable DER (DEFORM)

Chen et al. (2024, CoRL) [19] present a fully differentiable Discrete Elastic Rod implementation that enables end-to-end gradient-based learning through the implicit DER solve. This is the closest existing work to our DisMech problem and provides:

- Differentiable elastic energy computation
- Differentiable implicit Euler with adjoint-based backpropagation
- Parameter identification from trajectory data
- Code available (CoRL 2024)

DEFORM could serve as a differentiable replacement for DisMech, enabling direct policy

gradient computation through the physics simulation without a surrogate.

12.3 Py-DiSMech (2025)

Lahoti and Jawed (2025) [20] release a Python-native DDG simulator for rods and shells with explicit ML library hooks, penalty-based implicit contact, and modular energy definitions. This is a potential drop-in replacement for our C++ DisMech dependency, with native PyTorch integration for hybrid architectures.

13 Comprehensive Comparison

	Pure MLP Surrogate	**Warm-Start (Arch. A)**	**Gray-Box (Arch. B)**	**DEQ (Arch. D)**	**MENO**	**DisMech (Baseline)**
Integrator	None	Implicit	Implicit	Learned	MatExp	Implicit
NN calls / RL step	1	10	30–50	10–30	10	0
Modifies solver?	N/A	No	Yes	Replaces	N/A	—
Physics guaranteed?	No	Yes	Yes	Partial	Partial	Yes
Accuracy	∼95%	∼100%	∼99%	∼97%	∼96%	100%
Training data	1M trans.	100K steps	100K steps	100K steps	100K steps	—
Implementation	Simple	Simple	Medium	Hard	Medium	Existing
Throughput gain	17,000×	3–5×	1.5–2×	5–10×	5–10×	1×
Rollout stability	Low	Perfect	High	Medium	Medium	Perfect

Table 5: Comprehensive comparison of all hybrid architectures. "Accuracy" refers to single-step state prediction relative to ground truth. "Throughput gain" is relative to DisMech baseline (27 ms per implicit step in C++).

14 Literature Context

14.1 KNODE-Cosserat (Hsieh et al., 2024)

The original KNODE-Cosserat [1] targets a *tendon-driven continuum arm* (fixed-base cantilever, 10 segments, 3D). The NN corrects *spatial* derivatives in a shooting-method BVP solver. Key results: 87% average DTW improvement in simulation, 58.7% in real-world experiments. However, the architecture requires a shooting method at each timestep (15–50 NN evaluations per timestep), making it slower than our proposed approach.

Adaptation gap: Our snake robot has free-body boundary conditions, curvature (not tendon) actuation, RFT friction, and uses a Verlet integrator (not BVP shooting). The *concept* transfers directly; the *implementation* must be built from scratch.

14.2 DD-PINN for Cosserat MPC (Licher et al., 2025)

Domain-Decomposed PINNs [2] achieve $44{,}000\times$ speedup for Cosserat rod simulation by encoding the rod PDEs as loss constraints. The approach uses closed-form differentiation (no autodiff at inference) and satisfies initial conditions automatically. Open-source code available from the Habich et al. [3] companion paper (467× speedup on articulated soft robots with PINN surrogate for MPC at 47 Hz).

14.3 Augmented Neural ODE with Cosserat Prior (Kasaei et al., 2025)

Kasaei et al. [4, 5] propose coupled Augmented Neural ODEs (ANODEs) for tendon-driven continuum robots. The Shape-NODE integrates Cosserat rod theory as a structural prior; the Control-NODE optimizes policy via MPC. Achieves millimeter-level accuracy, outperforming pure Neural ODEs and RNNs.

14.4 Structure-Preserving Networks

CLPNets [7] learn Hamiltonian dynamics on Lie groups $SE(3)$ while preserving all Casimir invariants. Theoretically ideal for Cosserat rods (a discretized rod is a chain of $SE(3)$ bodies), but only validated on 2-body systems. Port-Hamiltonian neural networks [7b] guarantee energy-preserving dynamics in latent coordinates but have not been applied to rods.

14.5 Sim-to-Real Residual Learning (Gao et al., 2024)

Gao et al. [6] learn a residual force field on top of an FEM backbone for sim-to-real transfer of soft robots. Outperforms system identification by 60%. The architecture (physics backbone + learned force correction) is conceptually identical to our KNODE residual variant.

15 Decision Framework

15.1 When to Use Each Approach

- **Pure MLP surrogate** (current implementation): Best when throughput is the primary concern and single-step accuracy >99% is achievable. Preferred for short-horizon tasks.
- **KNODE (replace Stages 2b–c)**: Best when MLP surrogate accumulates unacceptable drift over multi-step rollouts. The physics backbone prevents catastrophic divergence.
- **KNODE (residual correction)**: Best when very high accuracy is needed with minimal training data. The simplified physics handles the bulk of the dynamics; the NN only patches errors. Highest implementation complexity.
- **DD-PINN**: Best for offline surrogate training where inference speed >1,000× is required and the Cosserat PDEs are well-characterized. Requires PINN expertise.

15.2 Recommended Path

1. **Start with pure MLP surrogate** (already implemented). Evaluate multi-step rollout accuracy.

2. **If rollout drift is unacceptable** (>5% state error at episode end):

   (a) **Architecture A (warm-start)**: Lowest risk. Train an MLP to predict Newton initial guesses from DisMech rollout data. No solver modifications needed. Expected 3–5× speedup.

   (b) **Architecture B (gray-box residual)**: If unmodeled physics is the bottleneck (e.g., contact, friction). Inject learned force corrections into the Newton residual with autodiff Jacobians.

3. **If maximum speed with physics structure is needed**:

   (a) **Architecture D (DEQ)**: Replace Newton entirely with a learned fixed-point iteration. Highest throughput (5–10×) but requires careful Lipschitz constraint enforcement.

   (b) **MENO (matrix exponential)**: Decompose into linear (A-stable matrix exponential) + nonlinear (NN) parts. Natural for elastic rod stiffness.

4. **If differentiability through the simulator is needed** (for gradient-based policy optimization):

   (a) Investigate DEFORM [19] or Py-DiSMech [20] as differentiable DER backends

   (b) Combine with short-horizon differentiable rollouts + RL value function estimation

References

[1] Jiahao, T.Z., Adolf, R., Sung, C., Hsieh, M.A. (2024). "KNODE-Cosserat: A Knowledge-based Neural ODE Framework for Cosserat Rod-based Soft Robots." *arXiv:2408.07776*.

[2] Licher, J., Bartholdt, M., Krauss, H., Habich, T.-L., Seel, T., Schappler, M. (2025). "Adaptive Model-Predictive Control of a Soft Continuum Robot Using a Physics-Informed Neural Network Based on Cosserat Rod Theory." *arXiv:2508.12681*.

[3] Habich, T.-L., Mohammad, A., Ehlers, S.F.G., Bensch, M., Seel, T., Schappler, M. (2025). "Generalizable and Fast Surrogates: MPC of Articulated Soft Robots using PINNs." *arXiv:2502.01916*. Code: `tlhabich.github.io/sponge/pinn_mpc`.

[4] Kasaei, M., Alambeigi, F., Khadem, M. (2025). "A Synergistic Framework for Learning Shape Estimation and Shape-Aware Whole-Body Control Policy for Continuum Robots." *arXiv:2501.03859*.

[5] Kasaei, M., Ghobadi, M., Khadem, M. (2025). "Shape-Aware Whole-Body Control for Continuum Robots." *arXiv:2510.12332*.

[6] Gao, J., Michelis, M.Y., Spielberg, A., Katzschmann, R.K. (2024). "Sim-to-Real of Soft Robots with Learned Residual Physics." *IEEE RA-L. arXiv:2402.01086*.

[7] Eldred, C., Gay-Balmaz, F., Putkaradze, V. (2024). "Coupled Lie-Poisson Neural Networks." *arXiv:2408.16160*.

[8] "SoRoLEX: Learned Environments for Soft Robot RL." (2024). *arXiv:2410.18519*.

[9] Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D. (2018). "Neural Ordinary Differential Equations." *NeurIPS 2018* (Best Paper). *arXiv:1806.07366*.

[10] Wang, X., Realmuto, J. (2026). "Physics-Embedded Neural ODEs for Learning Antagonistic Pneumatic Artificial Muscle Dynamics." *arXiv:2602.23670*.

[11] Zheng, H., Sukhija, B., Li, C., Iten, K., Krause, A., Katzschmann, R.K. (2025). "Learning Soft Robotic Dynamics with Active Exploration." *arXiv:2510.27428*.

[12] Briden, J., Choi, C., Yun, K., Linares, R., Cauligi, A. (2024). "Constraint-Informed Learning for Warm Starting Trajectory Optimization." *arXiv:2312.14336*.

[13] Banerjee, S., Cauligi, A., Pavone, M. (2025). "Deep Learning Warm Starts for Trajectory Optimization on the ISS." *arXiv:2505.05588*.

[14] Agarwal, A., Ruan, Y., Pileggi, L. (2024). "A Hybrid Simulation of DNN-based Gray Box Models." *arXiv:2410.17103*.

[15] Chen, J. (2024). "Graph Neural Preconditioners for Iterative Solutions of Sparse Linear Systems." *arXiv:2406.00809* (ICLR 2025).

[16] Burger, A., Thiede, L., Aspuru-Guzik, A., Vijaykumar, N. (2025). "DEQuify: Deep Equilibrium Force Fields for Molecular Dynamics." *arXiv:2509.08734*.

[17] Akhare, D., Du, P., Luo, T., Wang, J.-X. (2025). "Im-PiNDiff: Implicit Neural Differential Model for Spatiotemporal Dynamics." *arXiv:2504.02260*.

[18] Zanardi, I., Venturi, S., Panesi, M. (2025). "MENO: Hybrid Matrix Exponential-based Neural Operator for Stiff ODEs." *arXiv:2507.14341*.

[19] Chen, Y., Zhang, Y., Brei, Z., Zhang, T., Chen, Y., Wu, J., Vasudevan, R. (2024). "DEFORM: Differentiable Discrete Elastic Rods." *CoRL 2024. arXiv:2406.05931*.

[20] Lahoti, R., Chaiyakul, R., Jawed, M.K. (2025). "Py-DiSMech: Scalable DDG Framework for Soft Robots." *arXiv:2512.09911*.

[21] Margenberg, N., Mehlmann, C. (2025). "Hybrid NN-FEM for Viscous-Plastic Sea-Ice." *arXiv:2512.09118*.